

NATIONAL UNIVERSITY OF COMPUTER & EMERGING SCIENCES

FAST School of Computing — Islamabad Campus

ASSIGNMENT 01**1. Rules****1.1 What you may use and nothing else**

Primitive types · pointers, including pointer-to-pointer · arrays, 1D, 2D and jagged · structures · the . and -> operators · new, new[], delete, delete[] · pass-by-value · pass-by-reference. Standard headers permitted for this assignment: <iostream> and <cstdio>.

1.2 Hard bans

None of the following may appear anywhere in your submission not in a helper, not in an unused function, not in commented-out code.

Classes, inheritance, polymorphism, or other object-oriented design; any member function inside a struct	Constructors, destructors, this
Operator overloading	Templates
The STL in any form — <vector>, <map>, <set>, <list>, <algorithm>, <string>, <memory>, <utility>	std::string, std::string_view
Smart pointers — unique_ptr, shared_ptr, make_unique	Exceptions — try, catch, throw
Lambdas, std::function, auto for deduced container types	Linked lists, trees, hash tables
File I/O — <fstream>, ifstream, ofstream, FILE*	goto
<cstring> / <string.h> and every function in them — strlen, strcpy, strcmp, strcat, memcpy, memset	malloc, calloc, realloc, free
VARIABLES AT FILE SCOPE (local) OF ANY KIND — global variables are banned outright	static local variables

**There
are no
global**

variables in this assignment.

Every assignment-defined parameter constant is a MACRO, never a variable; P1, P2, P3, and SEED must be derived from ROLL_N using the supplied macros.

Every lookup table — names, service codes, prices, day names — is declared INSIDE the function that uses it. A table at file scope is a global variable and is banned.

A banned construct inside a question scores zero for that question. A class, std::string, any STL container or any <cstring> function anywhere in the submission scores zero for the whole assignment.

1.3 Submission

One zip file named <ROLLNO>_A01.zip — for example 25i-3012_A01.zip — containing EXACTLY

these four files and nothing else:

Q1.h Q1.cpp Q2.h Q2.cpp

- No .exe, no .o, no build folder, no IDE project, and no report or other supplementary document. Submit exactly the four required source/header files.
- Late submissions follow the course policy. A corrupt or unopenable archive is a non-submission.

2. Your Personal Parameters

Let N be the numeric part of your roll number, leading zeros dropped. Roll 25i-3012 gives $N = 3012$.

Symbol	Formula	Meaning
P1	$(N \% 4) + 3$	Initial capacity of the Q1 fleet array
P2	$(N \% 3) + 2$	Capacity of an inner array the FIRST time it is created (both questions)
P3	$(N \% 5) + 1$	Slots added each time an inner array grows after that (both questions)
SEED	$N \% 100$	Drives the required starting data of both questions

Each of Q1.h and Q2.h must begin with the following compile-time ROLL_N definition pattern, followed by the supplied derived macros:

```
#ifndef ROLL_N
#define ROLL_N
3012

#endif/* your own N goes here */
#define P1 (((ROLL_N) % 4) + 3)
#define P2 (((ROLL_N) % 3) + 2)
#define P3 (((ROLL_N) % 5) + 1)
#define SEED ((ROLL_N) % 100)
```

Hard-coding the resulting numbers instead of computing them scores zero for §2 compliance. The headers must use the supplied ROLL_N definition pattern so the instructor can rebuild with DROLL_N=<another value> and every derived number changes with it.

2.1 Worked example — roll 25i-3012

$N = 3012$

$$3012 = 4 \cdot 753 + 0 \rightarrow 0 + 3 \rightarrow P1 = 3$$

$$P1 = (3012 \% 4) + 3$$

$$P2 = (3012 \% 3) + 2 \quad 3012 = 3 \cdot 1004 + 0 \rightarrow 0 + 2 \rightarrow P2 = 2$$

$$P3 = (3012 \% 5) + 1 \quad SEED = 3012 \% 100 \quad SEED = 12$$

$$3012 = 5 \cdot 602 + 2 \rightarrow 2 + 1 \rightarrow P3 = 3 \rightarrow$$

So this student's fleet array starts with room for 3 probes and doubles: $3 \rightarrow 6 \rightarrow 12$. Any inner array (a probe's telemetry log, a day's appointment slots) is created with 2 slots and then grows by 3: $2 \rightarrow 5 \rightarrow 8$. The

outer and inner arrays use different growth policies on purpose. Every sample output in this handout uses these values.

3. Marks

Part	Component	Marks	Subtotal
Q1	Easy — sizes, padding, hand-written string helpers	10	
	Medium — building and querying the fleet	20	
	Hard — ownership, copying, removal, merge	20	50
Q2	Easy — sizes, padding, pointer arithmetic over a raw array	10	
	Medium — the jagged week, booking, cancelling	20	
	Hard — moving, the pointer index, teardown	20	50
	TOTAL	100	100

Code-only submission: grading is based on compilation, required output, behavior, pointer ownership, and memory safety. No report or supplementary written submission is required. Intentional by-value exception: leaks caused solely by command 7 (Q1) and command 12 (Q2) are exempt from leak deductions. All other leaks and memory errors are graded normally.

Page 2 of 12

FAST-NUCES Islamabad | CS2001 Data Structures | Assignment 01

QUESTION 1 — Deep Space Mission Control (50 marks)

Mission control tracks a fleet of probes. Each probe has a call sign of its own length and a telemetry log that grows as readings arrive. All dynamically allocated data belonging to the fleet lives on the heap; the Fleet objects themselves are ordinary local variables in main.

Structures — put these in Q1.h exactly as written

```

struct Reading {
    sensor[9]; float    stored IN PLACE // 'N'
    value; char status;
};

char
    // 8 chars + '\0',    nominal, 'W' warning,
    'C' critical

    struct Probe {
        int probeId;

        char* callSign; // heap block, exactly
        length + 1 Reading* readings; // heap
    ARRAY OF STRUCT VALUES int
        int readingCount;
        int readingCapacity; };
    struct Fleet {
        capacity;
    };

    // heap array of
    Probe** probes; int POINTERS to Probe
    count; int

```

```

Fleet f -----> Probe*[capacity] |
p0 | p1 | p2 | null |
|
v
Probe

#define
SIGN_LIMIT
20

#define SENSOR_LIMIT 8
#define MAX_PROBES 4096
#define MAX_READINGS 64
#define ID_MIN 1
#define ID_MAX 9999

```



STACK HEAP

```

one new[] / delete[] <--

```

```

| \
char[n+1] Reading[readingCapacity] <-- new[]/delete[]

```

<-- one new / delete each

Three levels, three lifetimes. Releasing the pointer array does not touch a single Probe; releasing a Probe does not touch the array. A Reading is a value living inside its array — it owns nothing, and copying one copies everything it contains.

Q1-A Easy (10 marks)

In this part the subscript operator [] must not appear in any EXPRESSION. It may appear only in a declaration (char buf[64];). Use pointer arithmetic and dereferencing. One subscripted expression scores 0 for Q1-A.

```

int myStrLen(const char* s);
void myStrCopy(char* dest, const char* src);
int myStrCompare(const char* a, const char* b);
char* cloneCString(const char* src);
void reportSizes();

```

myStrLen	Characters before the terminating '\0'. A null argument gives 0.
myStrCopy	Copies src into dest INCLUDING the terminator. dest is assumed large enough.
myStrCompare	Advance both while the characters match and neither is '\0', then return (int)(unsigned char)*p – (int)(unsigned char)*q. Equal gives exactly 0; ("abc","ab") gives 99; ("abc","abd") gives –1.
cloneCString	Allocates a NEW block of exactly myStrLen(src)+1 chars and copies src into it. nullptr for a null argument. OWNERSHIP PASSES TO THE CALLER — nothing in the return type says so.
reportSizes	Prints the four lines below. payload is the sum of sizeof of the members; padding is computed, never typed as a literal. ptr means sizeof(char*).

```

"sizeof char=%d int=%d float=%d ptr=%d\n"
"sizeof Reading=%d payload=%d padding=%d\n"
"sizeof Probe=%d payload=%d padding=%d\n"
"sizeof Fleet=%d payload=%d padding=%d\n"

```

Q1-B Medium (20 marks)

```
void initFleet(Fleet& f, int initialCapacity);
bool growFleet(Fleet& f);
bool addProbe(Fleet& f, int probelId, const char* callSign);
bool growReadings(Probe* p);
bool addReading(Probe* p, const char* sensor, float value, char status);
probe* findProbe(const Fleet& f, int probelId);
probe** findSlot(const Fleet& f, int probelId);
float probeHealth(const Probe* p);
void printProbe(const Probe* p);
void printFleet(const Fleet& f);
```

initFleet	Allocates the pointer array, sets every slot to nullptr, count = 0, capacity = the requested value (below 1 is treated as 1).
growFleet	New capacity is capacity × 2, except that 0 becomes P1. Above MAX_PROBES: print ERR FLEET_FULL, return false, change nothing. Otherwise allocate, null every slot, transfer the first count pointers, release the old array, repoint, and update capacity. Count is not touched.
addProbe	Validate in this order and complete ALL of it before the first allocation: id in [ID_MIN, ID_MAX] else ERR BAD_ID; call sign length in [1, SIGN_LIMIT] else ERR BAD_SIGN; id not already present else ERR DUP_ID. Then grow if count == capacity. Then create the probe with a cloned call sign, readings = nullptr, readingCount = 0, readingCapacity = 0; store at index count and increment count.
growReadings	New capacity is readingCapacity + P3, except that 0 becomes P2. Above MAX_READINGS: print ERR LOG_FULL, return false, change nothing. Copy the existing readings FIELD BY FIELD.
addReading	p null → ERR NO_PROBE; sensor length outside [1, SENSOR_LIMIT] → ERR BAD_SENSOR; status not one of N W C → ERR BAD_STATUS; sensor already logged on this probe (compare with myStrCompare) → ERR DUP_SENSOR. In that order. Then grow if full, and append at index readingCount.
findSlot	The ADDRESS of the array element holding the match, so the caller can write through it. nullptr when there is no match, when count is 0, or when probes is nullptr.
findProbe	The matching Probe*, or nullptr. Must be implemented in terms of findSlot. Prints nothing, ever.
probeHealth	N counts 1.0, W counts 0.5, C counts 0.0. The result is their mean. A probe with no readings, or a null pointer, gives exactly 0.0 — never divide by zero.

```
printProbe: "PROBE %04d | %-20s | LOGS=%d | HEALTH=%.2f\n" then " (no telemetry)\n" if readingCount
== 0
    else one line per reading, i from 0:
        "<%02d> %-8s VAL=%.2f ST=%c\n"
    a null Probe* prints "ERR NO_PROBE\n" and nothing else

printFleet: "FLEET count=%d capacity=%d\n"
    then " (empty)\n" if count == 0, else printProbe for slots
    0 .. count-1 in that order
```

`then "END FLEET\n"`

Ordering is insertion order — the physical order of the pointer array. Never sort.

Q1-C Hard (20 marks)

```
void destroyProbe(Probe*& p);
void compactFleet(Fleet& f, int removedIndex);
bool removeProbe(Fleet& f, int probeld);
void deepCopyProbe(const Probe* src, Probe*& dest);
void aliasCopyProbe(Probe* src, Probe* dest);
bool mergeFleets(Fleet& target, const Fleet& source);
void destroyFleet(Fleet& f);
```

Page 4 of 12

FAST-NUCES Islamabad | CS2001 Data Structures | Assignment 01

```
bool addProbeByValue(Fleet f, int probeld, const char* callSign);
void loadFleetA(Fleet& f);
void loadFleetB(Fleet& f);
```

destroyProbe	Releases the call sign, then the telemetry array, then the Probe, then sets the caller's pointer to nullptr. A null argument does nothing. Note the parameter type — it is a reference to a pointer, and that is load-bearing.
compactFleet	Closes the hole at removedIndex by moving every later pointer one slot toward the front, nulls the slot that becomes unused, and decrements count. Relative order is preserved. An out-of-range index does nothing. Frees nothing — the caller has already torn that probe down.
removeProbe	Locate with findSlot; no match → ERR NOT_FOUND, false. Otherwise record the index, destroy the probe completely, then compact. capacity never shrinks.
deepCopyProbe	The caller must pass dest == nullptr. Allocates the new Probe INSIDE the function and returns it through dest. The call sign gets its own block; ifsrc->readingCapacity > 0, allocate a new Reading array of exactly that capacity and copy the first readingCount readings field by field; if it is 0, dest->readings must be nullptr. A null src gives a null dest. Destroying src and *dest in either order must both succeed and neither may affect the other.
aliasCopyProbe	Assigns all five fields of *src to *dest directly. Allocates nothing. Does nothing if either pointer is null. Implement it exactly as described and do not make it safe. The driver intentionally uses it to demonstrate shallow copying.
mergeFleets	For each probe of source in order, if target has no probe with that id, add a copy of it — call sign and every reading. Probes already in target are skipped silently. Return false the moment a nested add fails. mergeFleets(f, f) must terminate, return true, and leave f with exactly the count and capacity it had on entry. A nested failure may print its own specified error; mergeFleets must not print a duplicate low-level error.
destroyFleet	Destroys every probe, releases the pointer array, then leaves the SAFE RESET STATE: probes = nullptr, count = 0, capacity = 0. Calling it twice is harmless. A destroyed fleet printed with printFleet produces exactly: "FLEET count=0 capacity=0\n", " (empty)\n", "END FLEET\n".

addProbeByValue	The body must be character-for-character identical to addProbe. The parameter is the only difference: Fleet& becomes Fleet. Do not make it safe or add cleanup not present in addProbe.
-----------------	---

Required starting data (loadFleetA / loadFleetB) Each loader first destroys the current target structure and then reinitializes it before loading the required data. Therefore repeated SEED_A or SEED_B commands are safe and deterministic.

```
SIGNS[8] = {"Voyager", "Pathfinder", "Odyssey", "Horizon",
           "Sentinel", "Aurora", "Vanguard", "Meridian"}
SENSORS[6] = {"TEMP-A", "PWR-BUS", "RAD-CNT", "GYRO-X", "COMMS-1", "FUEL-P"}
STATUS[3] = "NWC" (declare all three INSIDE the function)

FLEET A: total = (SEED % 3) + 3 probes, i = 0 .. total-1
    id(i) = 1000 + SEED + 11*i
    sign(i) = SIGNS[(SEED + i) % 8]
    logs(i) = (SEED + i) % 4 readings, j = 0 .. logs(i)-1
        sensor = SENSORS[(SEED + i + j) % 6]
        value = (float)(SEED + 10*i + 3*j) + 0.5f
        status = STATUS[(SEED + 2*i + j) % 3]

FLEET B: probe 1000+SEED "Relay-Alpha" 1 reading
    SENSORS[SEED % 6], (float)SEED + 0.5f, STATUS[SEED % 3]
    probe 8000+SEED "Relay-Beta" 2 readings
        j=0: SENSORS[(SEED+1)%6], (float)SEED+20.5f, STATUS[(SEED+1)%3]
        j=1: SENSORS[(SEED+2)%6], (float)SEED+40.5f, STATUS[(SEED+2)%3]
```

For SEED = 12 that is three probes 1012 with no telemetry at all, 1023 with one reading, 1034 with two and fleet B’s first probe deliberately collides with fleet A’s first. Note that probe 1012 has zero readings: your code must survive that everywhere.

4.4 The by-value task (assessed through command 7 and memory-safety tests) The by-value task is assessed directly through command 7. Keep addProbe and addProbeByValue exactly as specified; do not modify either function to repair the intentional pass-by-value behavior. The driver exposes both spare-capacity and no-spare-capacity cases through instructor tests.

4.5 Q1 driver/Menu

After the two startup lines, read whitespace-separated commands from standard input until command 0 or end of input. An unknown number prints ERR BAD_CMD and the loop continues. Initialize Fleet A with capacity P1, Fleet B with capacity 2, and the clone pointer as nullptr.

Your code should have the following options available to run in a menu based output.

```
"=== MISSION CONTROL ===\n"
"ROLL N=%d P1=%d P2=%d P3=%d\n"
```

#	Command	Effect
0	EXIT	Leave the loop.
1	ADD id sign	addProbe on A; on true print "OK PROBE_ADDED %04d\n".
2	LOG id sensor value status	findProbe on A; null → ERR NOT_FOUND; else addReading, and on

		true print "OK LOG_ADDED %04d %s\n" with id and sensor.
3	REMOVE id	removeProbe; on true print "OK PROBE_REMOVED %04d\n".
4	FIND id	null → ERR NOT_FOUND; else "OK FOUND %04d\n" then printProbe.
5	PRINT_A	Print "A:\n" then printFleet(A).
6	GROW	growFleet(A); on true print "OK FLEET_GROWN capacity=%d\n".
7	ADD_BYVAL id sign	Print "BYVAL pre count=%d capacity=%d\n"; call addProbeByValue and keep the result; print "BYVAL returned=%d\n" (1 or 0); print "BYVAL post count=%d capacity=%d\n".
8	CLONE id	null → ERR NOT_FOUND; else destroyProbe the current clone, deepCopyProbe into it, print "OK CLONED %04d\n".
9	PRINT_CLONE	Null clone → ERR NO_CLONE; else "CLONE:\n" then printProbe.
10	ALIAS id	See below.
11	SEED_A	loadFleetA(A) first destroys and reinitializes A, then prints "OK SEED_A count=%d\n".
12	SEED_B	loadFleetB(B) first destroys and reinitializes B, then prints "OK SEED_B count=%d\n".
13	PRINT_B	Print "B:\n" then printFleet(B).
14	MERGE_B	Record A.count, mergeFleets(A, B); on true "OK MERGED added=%d count=%d capacity=%d\n" with added = the change in A.count; on false "ERR MERGE_FAILED\n".
15	MERGE_SELF	Merge A with itself. It must terminate, return true, and leave A with exactly the count and capacity it had on entry.
16	DESTROY	destroyFleet(A) then print "OK FLEET_DESTROYED\n". B and the clone are not destroyed by this command.
17	SIZES	reportSizes().

Command 10 in detail Look the probe up in A; not there → ERR NOT_FOUND. Otherwise declare a Probe on the STACK with all five fields zeroed; call aliasCopyProbe with the found probe assrc and the address of the stack Probe as dest; then print the required ALIAS lines. Do not release anything belonging to the stack Probe.

"ALIAS sign=%d logs=%d\n" 1
when the copy holds the SAME
address as



the source in that field, 0 otherwise
then, if the copy's call sign is non-null and non-empty, write 'Z' over
its first character THROUGH THE COPY; if the copy's reading array is
non-null and readingCount > 0, set the copy's reading 0 status to 'C'
and read the SOURCE's reading 0 status into a char st, else st = '-';
"ALIAS after sign=%s log0=%c\n" using the SOURCE's call sign and st

EveryQ1 error string

"ERR_BAD_ID\n" "ERR LOG_FULL\n" "ERR_BAD_CMD\n"
NO_PROBE\n" "ERR

```

"ERR DUP_SENSOR\n"
"ERR NOT_FOUND\n"
"ERR NO_CLONE\n"
"ERR \"ERR MERGE_FAILED\n\" and the final line
BAD_SIGN\n\" \"ERR DUP_ID\n\" of every run: \"BYE\n\"
"ERR FLEET_FULL\n"

"ERR BAD_SENSOR\n\" \"ERR BAD_STATUS\n\"

```

A bool function prints the ERR line specified for the error it directly detects and prints nothing when it returns true. A composite function may propagate a nested failure without printing a second copy of the same low-level error; the driver prints the required OK line only on success. There are no other output strings in the program.

Shutdown: after command 0 or end-of-input, destroy the clone if non-null, destroy Fleet A, destroy Fleet B, print BYE, and return 0. This guarantees the normal Q1 program shutdown releases all owned allocations.

QUESTION 2 — Glow & Grace Salon Appointment Book (50 marks)

A salon books appointments across a seven-day week. This is a **different memory shape from Q1**: the outer array holds STRUCTS, not pointers, and each of those structs owns an inner array of its own length a jagged 2-D layout. Do not assume anything you learned in Q1 transfers unchanged.

Structures — put these in Q2.h exactly as written

```

struct Appointment {
    char*
    // heap block, exactly length
    + 1 // 8 chars + '\0', stored
    IN PLACE

    clientName; char service[9];
    float price;
}; struct DaySchedule {
    // heap ARRAY OF STRUCT
    VALUES
    Appointment* slots; int
    int count;
}; struct Week {
    capacity;

#define DAYS_IN_WEEK 7
#define NAME_LIMIT 14
#define SERVICE_LIMIT 8
#define MAX_SLOTS 40
#define CLIENT_MIN 1

#define CLIENT_MAX 9999
STACK HEAP

Week w ----> DaySchedule[7]

<-- one new[] / delete[] [0].slots
-> Appointment[c0] <-- one new[]/delete[] per day, [1].slots
-> Appointment[c1] each a DIFFERENT length
[2].slots -> nullptr (a day with no bookings)
...

```

and every Appointment in every slot array owns a char[n+1]

Tearing the week down means, innermost first: every name block, then every day's slot array, then the array of days. A day that was never booked has slots == nullptr and capacity 0.

Q2-A Easy (10 marks)

The [] rule of Q1-A applies to this part too. You will need myStrLen, myStrCopy, myStrCompare and cloneCString here as well; you may copy your Q1 implementations across, and they are not re-marked.

```
void fillPrices(float* begin, float* end, int n);  
float sumRange(const float* begin, const float* end);
```

Page 7 of 12

FAST-NUCES Islamabad | CS2001 Data Structures | Assignment 01

```
float* maxElementPtr(float* begin, float* end);  
int countAbove(const float* begin, const float* end, float threshold);  
void reverseInPlace(float* begin, float* end);  
void reportSizes();
```

fillPrices	Writes (float)((n + 613*i) % 4000) / 2.0f + 250.0f into the i-th element, i counted from begin. All ranges are half-open: begin is the first element, end is one past the last.
sumRange	Sum of the range. An empty range sumsto 0.
maxElementPtr	The address of the FIRST maximum; nullptr for an empty range.
countAbove	How many elements are strictly greater than threshold.
reverseInPlace	Reverses the range in place with two converging pointers.
reportSizes	Printsthe three SIZEOF lines below, using the same payload/padding rules as Q1-A.

"SIZEOF Appointment=%d payload=%d padding=%d\n"

"SIZEOF DaySchedule=%d payload=%d padding=%d\n"

"SIZEOF Week=%d payload=%d padding=%d\n"

Q2-B Medium (20 marks)

```
void initWeek(Week& w);  
bool growDay(DaySchedule& d);  
bool bookAppointment(Week& w, int day, int clientId, const char* name, const char* service, float price);  
bool cancelAppointment(Week& w, int day, int slot);  
Appointment* findAppointment(const Week& w, int clientId, int& outDay, int& outSlot);  
float dayRevenue(const DaySchedule& d);  
void printDay(const DaySchedule& d, int dayIndex);  
void printWeek(const Week& w);
```

initWeek	Allocates the array of 7 DaySchedule and sets every day to slots = nullptr, count = 0, capacity = 0. dayCount = DAYS_IN_WEEK. No day owns a slot array until
----------	--

	something is booked into it.
growDay	New capacity is capacity + P3, except that 0 becomes P2. Above MAX_SLOTS: print ERR DAY_FULL, return false, change nothing. Copy the existing appointments field by field. Copying the clientName field copies a POINTER — think carefully about what that means for the old array, and what must NOT happen to it.
bookAppointment	If the Week is destroyed (days == nullptr), print ERR WEEK_DESTROYED and return false. Otherwise validate in this order: day in range -> ERR BAD_DAY; clientId in [CLIENT_MIN, CLIENT_MAX] -> ERR BAD_CLIENT; name length in [1, NAME_LIMIT] -> ERR BAD_NAME; service length in [1, SERVICE_LIMIT] -> ERR BAD_SERVICE; price in [100.0, 20000.0] -> ERR BAD_PRICE; same clientId already booked on that day -> ERR DUP_BOOKING. Finish all validation before the first allocation. Then grow if the day is full and append with a cloned name.
cancelAppointment	If the Week is destroyed (days == nullptr), print ERR WEEK_DESTROYED and return false. Otherwise bad day -> ERR BAD_DAY; slot outside [0, count-1] -> ERR BAD_SLOT. On success, release the name block, shift later appointments one slot toward the front, then decrement count. Relative order is preserved.
findAppointment	Searches all seven days in order, then slots in order. On a hit, writes the day and slot into the reference parameters and returns the Appointment*. On a miss or destroyed Week, writes -1 into both and returns nullptr.
dayRevenue	The sum of the prices of that day's live appointments. An empty day gives 0.0.

printDay For a live Week, print: "DAY %d %s | BOOKED=%d/%d | REVENUE=%.2f\n" with dayIndex, then the name from {"MON", "TUE", "WED", "THU", "FRI", "SAT", "SUN"} declared INSIDE the function, count, capacity, and dayRevenue. Then two-space "(free)\n" if count == 0, else one line per live slot: "<%02d> %04d %-14s %-8s PKR %8.2f\n". The driver must reject PRINT_DAY on a destroyed Week with ERR WEEK_DESTROYED.

printWeek For a live Week, print: "WEEK bookings=%d revenue=%.2f\n" with totals over all 7 days, then printDay for day 0 .. 6 in order, then "END WEEK\n". For a destroyed Week, print exactly: "WEEK bookings=0 revenue=0.00\n (destroyed)\nEND WEEK\n".

Q2-C Hard (20 marks)

```

bool moveAppointment(Week& w, int fromDay, int fromSlot, int toDay);
Appointment** buildIndex(const Week& w, int& outCount);
void sortIndexByPrice(Appointment** index, int n);
void printIndex(Appointment** index, int n);
void destroyIndex(Appointment**& index, int& n);
void destroyWeek(Week& w);
bool bookByValue(DaySchedule day, int clientId, const char* name, const char* service, float price);
void loadSeedWeek(Week& w);

```

moveAppointment	If the Week is destroyed (days == nullptr), print ERR WEEK_DESTROYED and return false. Otherwise: either day out of range -> ERR BAD_DAY; fromDay == toDay -> ERR SAME_DAY; fromSlot outside [0, count-1] -> ERR BAD_SLOT; same clientId already booked on destination day -> ERR DUP_BOOKING. Grow destination if full. The appointment is appended at the END of destination, removed from source by compaction, and its clientName pointer is transferred without duplication or release.
-----------------	--

buildIndex	Counts every live appointment, writes the total to outCount, and returns a NEW array of Appointment* in day order then slot order. Returns nullptr and writes 0 when the week is empty or destroyed. The caller owns only the index array; Appointments and clientName blocks remain owned by the Week.
sortIndexByPrice	Sorts the index into descending price order using any O(n ²) method. ONLY POINTERS MOVE. Appointments themselves are never relocated. Equal prices retain their current index order (stable ordering). A null index does nothing.
printIndex	"INDEX size=%d\n", then " (none)\n" if the index is null or n is 0, else one line per entry " [%02d] %04d %-14s %-8s PKR %8.2f\n", then "END INDEX\n".
destroyIndex	Releases the array of pointers and sets the caller's pointer to nullptr and the caller's count to 0. Both parameters are references for a reason.
destroyWeek	Releases every clientName, every day slot array, then the array of days. Leaves days = nullptr, dayCount = 0. Calling it twice is harmless. printWeek on a destroyed Week must print exactly: "WEEK bookings=0 revenue=0.00\n (destroyed)\nEND WEEK\n".
bookByValue	The successful-booking logic is the same as the tail of bookAppointment, but the DaySchedule parameter is passed by value. This intentionally demonstrates shallow copying; do not make it safe.

Required starting data (loadSeedWeek)

```
NAMES[8] = {"Ayesha", "Hina", "Sana", "Mahnoor", "Zara", "Iqra", "Nimra", "Rabia"}
SERVICES[6] = {"HAIRCUT", "FACIAL", "MANI", "PEDI", "COLOR", "MAKEUP"}
PRICES[6] = {1500.0f, 2500.0f, 800.0f, 4200.0f, 1200.0f, 3000.0f}
(declare all three INSIDE the function)
```

```
total = (SEED % 3) + 9 weekend bookings, k = 0 .. total-1
    day = 5 + (k % 2) 5 = SAT, 6 = SUN
    clientId = 500 + SEED + 7*k
    name = NAMES[(SEED + k) % 8]
    which = (SEED + k) % 6
    service = SERVICES[which] price = PRICES[which]
```

```
then ONE midweek booking:
    day = SEED % 5 clientId = 500 + SEED + 7*total
    name = NAMES[(SEED + total) % 8] which = (SEED + total) % 6
```

For SEED = 12 that is nine weekend bookings — five on Saturday, four on Sunday — plus one on Wednesday. Saturday therefore outgrows its first slot array, whatever your P2 and P3 are, and four days stay empty.

5.4 Q2 driver/Menu

Your code should have the following options available to run in a menu based

output. "=== GLOW & GRACE SALON ===\n"
"ROLL N=%d P2=%d P3=%d\n"

#	Command	Effect
---	---------	--------

0	EXIT	Leave the loop.
1	BOOK day id name service price	If an index is currently held, destroy it before attempting the booking. Then call bookAppointment; on true print "OK BOOKED d%d %04d\n".
2	CANCEL day slot	If an index is currently held, destroy it before attempting the cancellation. Then call cancelAppointment; on true print "OK CANCELLED d%d s%d\n".
3	PRINT_DAY day	Destroyed Week -> ERR WEEK_DESTROYED; otherwise out-of-range day -> ERR BAD_DAY; else printDay.
4	PRINT_WEEK	printWeek.
5	MOVE fromDay fromSlot toDay	If an index is currently held, destroy it before attempting the move. Then call moveAppointment; on true print "OK MOVED d%d s%d -> d%d\n".
6	BUILD_INDEX	Release any index you already hold, build a new one, print "OK INDEX_BUILT size=%d\n".
7	PRINT_INDEX	printIndex.
8	SORT_INDEX	sortIndexByPrice then "OK INDEX_SORTED\n".
9	DROP_INDEX	destroyIndex then "OK INDEX_DROPPED\n".
10	FIND id	Miss → ERR NOT_FOUND; else "OK FOUND %04d day=%d slot=%d %-8s PKR %8.2f\n".
11	SEED	Release any index already held, loadSeedWeek(w), then print "OK SEED_LOADED\n".
12	BOOK_BYVAL day id name service price	Day out of range -> ERR BAD_DAY. Destroyed Week -> ERR WEEK_DESTROYED. Otherwise print the BYVAL pre line, call bookByValue, print the return value, then the BYVAL post line. This command is assessed through automated tests.
13	DESTROY_WEEK	Release any index, destroyWeek, then print "OK WEEK_DESTROYED\n".
14	UTILS	reportSizes(), then declare float list[10], run fillPrices with n = 10, and print the five A2 lines below. The array has exactly 10 elements and n must be 10.

"A2 LIST:" then " %.1f" per element then a newline
"A2 SUM=%.2f\n" sumRange over the whole array
"A2 MAX=%.1f OFF=%d\n" the value, and the offset from begin
"A2 ABOVE=%d\n" countAbove with a threshold of 1000.0f
reverseInPlace, then "A2 REV:" then " %.1f" per element then a newline

Every Q2 error string "ERR BAD_DAY\n" "ERR BAD_SLOT\n" "ERR BAD_CLIENT\n" "ERR BAD_NAME\n"
"ERR BAD_SERVICE\n" "ERR BAD_PRICE\n" "ERR DUP_BOOKING\n" "ERR SAME_DAY\n" "ERR DAY_FULL\n"
"ERR WEEK_DESTROYED\n" "ERR NOT_FOUND\n" "ERR BAD_CMD\n" "BYE\n"

Shutdown: after command 0 or end-of-input, destroy any held index, destroyWeek(w), print BYE, and return 0. This guarantees the normal Q2 program shutdown releases all owned allocations.

Bound every token read: call std::cin.width(sizeof(buf)) immediately before each std::cin >> buf. The width applies to one extraction and then resets. Buffers of 64 bytes are enough.

Sample runs — roll 25i-3012

N = 3012, so P1 = 3, P2 = 2, P3 = 3, SEED = 12. The reference outputs below show the exact required structure, spacing, ordering, and wording. Your own run will differ in data where your parameters differ.

Q1 — seed, grow, log, clone, remove, merge with self

Input	11 5 1 4001 Kepler 2 4001 TEMP-A 15.50 N 4 4001 8 1034 9 3 1034 9 15 5 0
-------	---

=== MISSION CONTROL ===

ROLL_N=3012 P1=3 P2=2 P3=3



<01> 35.50 ST=C
GYRO-X
VAL= END
FLEET
OK



PROBE 1012 | PROBE_ADDED 4001 HEALTH=0.00 |
Sentinel (no
telemetry)
PROBE 1023 | LOGS=0 |
Aurora <00> 22.50 LOGS=1 |
PWR-BUS VAL= PROBE ST=C HEALTH=0.00 |
1034 | Vanguard 32.50 LOGS=2 |
<00> RAD-CNT VAL= ST=W HEALTH=0.25

OK LOG_ADDED 4001 TEMP-A

OK FOUND 4001
PROBE 4001 | Kepler | LOGS=1 | HEALTH=1.00 <00> TEMP-A
VAL= 15.50 ST=N



Vanguard <00> ST=W 35.50 ST=C
RAD-CNT VAL=
<01> GYRO-X VAL= | LOGS=2 |
OK PROBE_REMOVED HEALTH=0.25
1034 CLONE:

PROBE 1034 | 32.50

PROBE 1034 | Vanguard <00> LOGS=2 | HEALTH=0.25
RAD-CNT VAL=
32.50 ST=W

<01> GYRO-X VAL= 35.50 ST=C OK
MERGED added=0 count=3 capacity=6
A: FLEET count=3 capacity=6



Aurora <00>
PWR-BUS VAL=
PROBE 4001 |
Kepler <00>

TEMP-A VAL= END
 FLEET
 BYE

22.50

ST=C 15.50 ST=N

HEALTH=0.00 |
 LOGS=1 |
 HEALTH=1.00

| LOGS=0 |
 HEALTH=0.00 |
 LOGS=1 |



Q2 — seed, move between days, build and sort the index, cancel

Input	11 3 5 5 5 0 1 3 5 3 1 6 8 7 2 1 0 3 1 9 0
-------	--

=== GLOW & GRACE SALON ===

ROLL_N=3012 P2=2 P3=3

OK SEED_LOADED

DAY 5 SAT | BOOKED=5/5 | REVENUE=5800.00

<00> 0512 Zara HAIRCUT PKR 1500.00

Page 11 of 12

FAST-NUCES Islamabad | CS2001 Data Structures | Assignment 01

<01> 0526 Nimra MANI PKR 800.00 <02> 0540 Ayesha

COLOR PKR 1200.00 <03> 0554 Sana HAIRCUT PKR

1500.00 <04> 0568 Zara MANI PKR 800.00 OK MOVED d5

s0 -> d1

DAY 5 SAT | BOOKED=4/5 | REVENUE=4300.00 <00> 0526

Nimra MANI PKR 800.00 <01> 0540 Ayesha COLOR PKR

1200.00 <02> 0554 Sana HAIRCUT PKR 1500.00 <03>

0568 Zara MANI PKR 800.00

DAY 1 TUE | BOOKED=1/2 | REVENUE=1500.00 <00> 0512

Zara HAIRCUT PKR 1500.00 OK INDEX_BUILT size=10

OK INDEX_SORTED

INDEX size=10

[00] 0575 Iqra PEDI PKR 4200.00 [01] 0533 Rabia PEDI

PKR 4200.00 [02] 0547 Hina MAKEUP PKR 3000.00 [03]

0519 Iqra FACIAL PKR 2500.00 [04] 0561 Mahnoor FACIAL

PKR 2500.00 [05] 0512 Zara HAIRCUT PKR 1500.00 [06]

0554 Sana HAIRCUT PKR 1500.00 [07] 0540 Ayesha

COLOR PKR 1200.00 [08] 0526 Nimra MANI PKR 800.00

[09] 0568 Zara MANI PKR 800.00 END INDEX

OK CANCELLED d1 s0

DAY 1 TUE | BOOKED=0/2 | REVENUE=0.00

(free)

OK INDEX_DROPPED

BYE

